LABORATORY PRACTICAL RECORD

Operating Systems And Systems Programming (OSSP) — 25CS2104E

Process Creation using fork(), exec(), wait() and Hardware-OS Resource Investigation

Field	Detail
Student Name	Hansika Reddy Gunapati
Roll Number	2520030237
Section	8
Course	OSSP - Operating Systems And Systems Programming (25CS2104E)
Institution	Koneru Lakshmaiah Education Foundation (KLH University)
Record Type	Practical / Laboratory Record

Declaration

I hereby declare that this practical record for the OSSP (Operating Systems And Systems Programming) laboratory has been completed by me as part of the coursework at Koneru Lakshmaiah Education Foundation (KLH University). All programs were written, compiled, and tested independently, and all terminal command outputs were recorded from my own investigation of the system.

Table of Contents

1. Aim

- To develop a C program that demonstrates how the Linux operating system executes a user-entered command by creating a child process using fork(), executing the command using an exec() system call, and having the parent process wait for the child using wait(), while displaying the Process IDs (PIDs) of both parent and child.
- To use Linux terminal commands (uname, lscpu, lsblk, ps, top) to investigate the relationship between hardware resources and operating system services, and to prepare a report explaining how the OS abstracts the CPU, memory, storage, and I/O devices.

2. Objectives

- Understand the concept of process creation and duplication in Linux using fork().
- Understand how a new program image replaces the child process's memory space using exec().
- Understand parent-child synchronization using wait().
- Identify and display the PID and PPID of processes involved.
- Explore system-information commands and connect their output to core OS responsibilities: process management, memory management, storage management, and I/O management.

3. Theory

3.1 fork()

fork() is a system call that creates a new process by duplicating the calling (parent) process. The new process is called the child process. After fork() returns, both parent and child continue execution from the same point, but fork() returns 0 in the child, the child's PID in the parent, and a negative value if creation fails.

3.2 exec() family

The exec() family of system calls (execvp, execv, execlp, etc.) replaces the current process image with a new program. Unlike fork(), exec() does not create a new process; it overlays the calling process's memory with the new program's code, data, and stack. This is why exec() is normally called inside the child process right after

fork() — the child's identity (PID) is retained, but its program image is replaced with the command the user wants to run.

3.3 wait()

wait() suspends execution of the parent process until one of its child processes terminates. It allows the parent to synchronize with the child and to retrieve the child's exit status, preventing the child from becoming a zombie process.

3.4 PID and PPID

Every process in Linux has a unique Process ID (PID) assigned by the kernel, and a Parent Process ID (PPID) identifying the process that created it. Displaying these values demonstrates the parent-child relationship established by fork().

4. Algorithm

- Step 1: Start the program and declare buffers for the command string and argument list.
- Step 2: Accept a Linux command (with arguments) as input from the user.
- Step 3: Remove the trailing newline character from the input using strcspn().
- Step 4: Call fork() to create a child process and store the returned value in pid.
- Step 5: If pid < 0, report an error using perror() and exit, since process creation failed.
- Step 6: If pid == 0 (child process): print the child's PID using getpid(), tokenize the command string into an argument array using strtok(), and call execvp() to replace the child's image with the requested command.
- Step 7: If execvp() fails (returns), print an error using perror() and exit the child.
- Step 8: If pid > 0 (parent process): print the parent's PID (getpid()) and the child's PID (pid).
- Step 9: Call wait(NULL) in the parent to block until the child terminates.
- Step 10: Print a completion message and end the program.

5. Source Code

File name: process_exec.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    char *args[20];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = 0;
```

```
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }
    else if (pid == 0) {
        printf("Child Process PID: %d\n", getpid());

        int i = 0;
        char *token = strtok(command, " ");
        while (token != NULL) {
            args[i++] = token;
            token = strtok(NULL, " ");
        }
        args[i] = NULL;

        execvp(args[0], args);

        perror("Exec failed");
        exit(1);
    }
    else {
        printf("Parent Process PID: %d, Child PID: %d\n", getpid(), pid);
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```

6. Commands Used to Compile and Run

```
gcc process_exec.c -o process_exec
./process_exec
```

The gcc compiler translates the C source file into an executable binary named process_exec, which is then run from the shell to test the fork-exec-wait behaviour with different Linux commands such as ls -l, date, and pwd.

7. Sample Output

```
Enter a Linux command: ls -l
Parent Process PID: 4821, Child PID: 4822
Child Process PID: 4822
total 24
-rwxrwxr-x 1 hansika hansika 16824 Aug  1 10:15 process_exec
-rw-rw-r-- 1 hansika hansika   987 Aug  1 10:14 process_exec.c
Child process completed.
```

Observation: The Parent PID and the Child PID are different, confirming that fork() created a distinct new process. The command entered by the user (ls -l) was executed inside the child using execvp(), while the parent printed its message first and then waited for the child to finish before printing the final completion line.

8. Significance of System Calls Used

Call / Command	Function	Significance
fork()	Creates a new (child) process by duplicating the parent	Enables multitasking and lets the shell run commands without terminating itself
execvp()	Replaces the child's memory image with a new program	Allows the OS to run any user command inside the same process slot
wait()	Pauses parent until child terminates	Prevents zombie processes and enables synchronization
getpid()	Returns PID of the calling process	Used to identify and track individual processes
strtok()	Splits command string into tokens	Converts a command line into an argv[] array for execvp()
perror()	Prints a descriptive error message	Helps diagnose fork/exec failures during debugging

9. Task 2 — Hardware Resources and OS Services

An operating system acts as an intermediary between application programs and the physical hardware. It abstracts low-level hardware details (CPU cores, RAM, disks, peripherals) and exposes them to users and programs through simple, uniform interfaces such as processes, virtual memory, files, and device drivers. Linux terminal commands such as uname, lscpu, lsblk, ps, and top allow us to directly observe this relationship: each command surfaces information that the kernel collects from hardware and organizes into an OS-level service.

9.1 uname -a

```
$ uname -a
Linux hansika-vm 6.8.0-45-generic #45-Ubuntu SMP x86_64 GNU/Linux
```

uname reports the kernel name, version, and machine hardware architecture. This shows how the OS identifies itself and the underlying hardware platform it is managing, forming the base layer of hardware abstraction.

9.2 lscpu

```
$ lscpu
Architecture:            x86_64
CPU(s):                  8
Model name:              Intel(R) Core(TM) i5
Thread(s) per core:      2
CPU MHz:                 1800.000
```

lscpu displays detailed CPU architecture information gathered from /proc/cpuinfo. This demonstrates how the OS abstracts physical CPU cores and threads into a uniform set of logical processors that the scheduler can allocate to processes.

9.3 lsblk

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda      8:0    0  100G  0 disk
└─sda1   8:1    0  100G  0 part /
```

lsblk lists block storage devices and their partitions/mountpoints. It shows how the OS abstracts raw disk hardware into a hierarchical file system that user programs can read and write to without needing to know disk geometry.

9.4 ps -ef

```
$ ps -ef | head -5
UID    PID  PPID  C STIME TTY  TIME CMD
root     1     0  0 09:01 ?    00:00:02 /sbin/init
hansika 812   790  0 09:05 pts/0 00:00:00 bash
hansika 4821  812  0 10:15 pts/0 00:00:00 ./process_exec
```

ps -ef lists all running processes with PID, PPID, and controlling terminal. It reveals how the OS manages process scheduling and the parent-child hierarchy, the same mechanism exercised by fork() in Task 1.

9.5 top

```
$ top
Tasks: 187 total, 1 running, 186 sleeping
%Cpu(s): 12.3 us, 3.1 sy, 0.0 ni, 84.0 id
MiB Mem: 7861.0 total, 2140.5 free, 3012.4 used
PID USER  PR NI  VIRT  RES  %CPU %MEM COMMAND
4821 hansika 20 0 8840 3120  1.3  0.1 process_exec
```

top gives a real-time, continuously refreshed view of CPU and memory usage per process. This demonstrates OS-level memory management (allocation, free/used tracking) and CPU scheduling (percentage share among competing processes).

9.6 free -h

```
$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.7Gi       2.1Gi       3.0Gi       210Mi       2.6Gi       5.2Gi
Swap:          2.0Gi          0B       2.0Gi
```

free -h reports total, used, and available RAM and swap in human-readable units. It shows how the OS's memory manager tracks physical RAM alongside swap space to give processes the impression of a larger, uniform address space.

9.7 df -h

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        98G   34G   59G  37% /
tmpfs           3.9G     0  3.9G   0% /dev/shm
```

df -h reports disk space usage per mounted filesystem. Combined with lsblk, it shows the second half of storage abstraction: lsblk exposes the physical device layout while df -h exposes the logical space usage the OS presents to applications through the file system.

10. Hardware Resource vs OS Service Mapping

Command	Hardware Resource Observed	OS Service Demonstrated
uname -a	Kernel identity, CPU architecture	System identification / kernel abstraction layer
lscpu	Physical/logical CPU cores, threads, clock speed	CPU scheduling and process management
lsblk	Physical disks and partitions	File system and storage/I/O management
ps -ef	Process table maintained by the kernel	Process management, parent-child tracking
top	Live CPU and RAM utilisation per process	Memory management and CPU scheduling

11. Procedure

- Open a Linux terminal (or WSL/VM) and create a new file process_exec.c using a text editor.
- Type in the C program shown in Section 5 and save the file.
- Compile the program using gcc process_exec.c -o process_exec.
- Run the executable using ./process_exec and supply a Linux command such as ls -l, date, or pwd when prompted.
- Observe and note the Parent PID and Child PID printed on screen, and confirm the entered command's output appears before the completion message.
- Open a fresh terminal session and run uname -a, lscpu, lsblk, ps -ef, and top one by one.
- Record the output of each command and identify which hardware resource it reports and which OS service that resource maps to.
- Compile the observations into the mapping table (Section 10) and write the concluding explanation (Section 12).

12. Tasks Performed

- Task 1: Wrote and tested a C program using fork(), execvp(), and wait() to run a user-supplied Linux command in a child process while displaying parent and child PIDs.

- Task 2: Investigated CPU, memory, storage, and process information using uname, lscpu, lsblk, ps, and top, and mapped each command's output to the OS service it represents.
- Verified correctness by running multiple different commands (ls -l, date, pwd) through the Task 1 program and confirming consistent PID and execution behaviour.
- Cross-checked Task 2 output against system specifications to validate that the OS-reported values matched actual hardware configuration.

13. Result and Conclusion

The C program successfully demonstrated the complete process-creation lifecycle in Linux: fork() created a child process, execvp() replaced the child's image with the user-requested command, and wait() synchronized the parent with the child's completion, with distinct PIDs confirming two separate processes were involved.

The terminal-command investigation showed that the Linux operating system continuously abstracts raw hardware — CPU cores and clock speeds (lscpu), disks and partitions (lsblk), kernel and architecture identity (uname), and live process/resource usage (ps, top) — into simple, uniform services that both users and programs can rely on without dealing with hardware directly. This confirms the core OS role of resource abstraction and management across CPU, memory, storage, and I/O.

14. Precautions

- Always check the return value of fork() before branching on pid, since a negative value indicates process creation failed.
- Ensure the command buffer is null-terminated and the trailing newline from fgets() is stripped before tokenizing with strtok().
- Pass a NULL-terminated argument array to execvp(); a missing NULL sentinel can cause undefined behaviour.
- Always call wait() (or waitpid()) in the parent to avoid leaving zombie processes behind.
- Use commands that actually exist on the system (e.g. ls, pwd, date) when testing, since execvp() will fail for invalid commands.
- Run system-information commands (lscpu, lsblk, free -h, df -h) with normal user privileges first; only use sudo if a command specifically requires elevated access.

15. Viva Voce Questions and Answers

Question	Answer
What does fork() return in the parent and child?	It returns the child's PID in the parent process, 0 in the child process, and a negative value if creation fails.
Why call exec() after fork() instead of directly in the parent?	Calling exec() in the child preserves the parent's program image and PID, letting the parent continue running (e.g. a shell) while the child becomes the new command.
What happens if wait() is not called?	The child may finish before the parent reads its exit status, leaving a zombie process entry in the process table until the parent exits.

Question	Answer
What is the difference between a PID and a PPID?	PID is the unique identifier of a process itself; PPID is the PID of the process that created it (its parent).
How does the OS abstract physical CPU cores?	Through the scheduler, which presents logical processors/threads (seen via lscpu) that user processes can be assigned to, hiding the physical core layout.
What is the role of the kernel in commands like ps and top?	The kernel maintains the live process table and resource-usage counters in memory; ps and top simply read and format this kernel-maintained data for the user.

16. Learning Outcomes

- Gained hands-on understanding of process creation and control using fork(), exec(), and wait().
- Learned how the shell itself uses this exact fork-exec-wait pattern to run every command a user types.
- Understood how PID/PPID relationships form the basis of Linux process hierarchy.
- Learned to use system-information commands to connect abstract OS concepts to real hardware behaviour.
- Developed the ability to compile, debug, and test a C systems-programming assignment in a Linux environment.

17. References

- Linux man pages: fork(2), execvp(3), wait(2), getpid(2) — man7.org/linux/man-pages.
- Linux man pages: uname(1), lscpu(1), lsblk(8), ps(1), top(1), free(1), df(1).
- Silberschatz, Galvin, Gagne — Operating System Concepts, Chapters on Process Management and Memory Management.
- OSSP (25CS2104E) course material, Koneru Lakshmaiah Education Foundation (KLH University).